

Práctica 1: Introducción a libpcap y Wireshark

Objetivos de la práctica

- Familiarizarse con la biblioteca libpcap: esta biblioteca nos permite capturar paquetes, filtrarlos, modificarlos y estudiarlos desde un programa Python. Puede interpretarse que Wireshark sirve como front-end de esta biblioteca*.
- Demostrar que se ha realizado el tutorial de Wireshark con provecho.

Introducción

Existen multitud de recursos en Internet para aprender a usar libpcap en detalle:

- [Packet Capture With libpcap and other Low Level Network Tricks de NAU's Computer Systems Engineering.](#)
- [Aprendiendo a programar con Libpcap de Alejandro Lopez Monge.](#)

Las referencias proporcionadas abordan el uso de la libpcap desde programas en lenguaje C. En las prácticas usaremos un *wrapper* para Python llamado RC1-libpcap que nos encapsulará el acceso a las funciones de la biblioteca libpcap que está escrita en C. Este *wrapper* respeta los nombres de funciones, argumentos, retornos y tipos de datos de la biblioteca original y puede usarse de manera equivalente.

Veamos primero las funciones básicas que nos ofrece RC1-libpcap.

(**IMPORTANTE** Por favor, lea el enunciado entero antes de empezar.)

Biblioteca RC1-Libpcap

Abrir un archivo pcap

Para abrir un archivo (traza) previamente capturado:

```
pcap_open_offline(fname,errbuf)
```

Donde

- *fname* es una cadena con el nombre del archivo .pcap que se desea abrir.
- *errbuf* es un *bytearray* donde se guardará un mensaje de error en caso de que algo haya fallado al abrir el fichero
- La función nos devuelve un descriptor al archivo .pcap o None/Excepción en caso de que haya algún error

Ejemplo: `p = pcap_open_offline('traza.pcap', errbuf)`

Abre para lectura el archivo traza.pcap. En caso de error, guarda el mensaje en el *bytearray* errbuf.

Capturar de un interfaz

Para abrir un interfaz para captura:

```
pcap_open_live(device, snaplen, promisc, to_ms, errbuf)
```

Donde

- *device* es una cadena con el nombre de la interfaz que se quiere abrir (eth0, ens33...).
- *snaplen* es la cantidad de bytes que se quieren guardar por cada paquete. Es útil cuando no nos interesa la carga útil del paquete y así reduciríamos el tamaño de la captura.
- *promisc* indica si queremos abrirla en modo promiscuo (*promisc=1*) o no (*promisc=0*).
- *to_ms* duración del *timeout* de lectura. Tiempo que se espera para leer varios paquetes en una misma transacción (*polling*).
- *errbuf* es un *bytearray* donde se guardará un mensaje de error en caso de que algo haya fallado al abrir la interfaz.
- La función nos devuelve un descriptor al archivo .pcap o None/Excepción en caso de que haya algún error

Para abrir una interfaz es necesario tener permisos de superusuario. En la VM facilitada simplemente **debemos usar sudo** seguido de la instrucción con la que se llama al programa (por ejemplo: `sudo python3 practica1.py`).

Ejemplo: `p = pcap_open_live('ens33', BUFSIZ, 0, 100, errbuf)`

Abre la interfaz ens33 en modo no promiscuo, capturando hasta BUFSIZ bytes del paquete, con un timeout de lectura de 100 ms. (puede que la VM le alerte sobre la imposibilidad de capturar tráfico de modo promiscuo en caso de modificar el tercer argumento, no es importante para la realización de las prácticas, acepte y continúe). En caso de error, guarda el mensaje en el `bytearray` `errbuf`.

Leer tráfico de archivo o interfaz

`pcap_loop(descr, cnt, callback, user)`

Donde:

- `descr` es el descriptor PCAP del que queramos leer (que anteriormente hemos abierto con `open_pcap_live` o `open_pcap_offline`).
- `cnt` es el número de paquetes a analizar (-1 para ilimitados).
- `callback` es una función de atención al paquete. Esta función se ejecutará por cada paquete leído o capturado.
- `user` es una variable auxiliar que sirve para pasar datos a la función de atención.
- `pcap_loop` nos devuelve:
 - 0 si se leyó la traza entera o se superó el límite `cnt`.
 - -1 Si hubo errores
 - -2 Si fue interrumpido por `pcap_breakloop()` (u otras).
 - Otros valores si se capturó un paquete.

Ejemplo: `ret = pcap_loop(descr, -1, procesa_paquete, None)`

Y a su vez: `callback(user, pkt_header, pkt_data)` # Según el caso anterior, nombraremos a esta función `procesa_paquete`, con la signatura indicada.

Donde:

- `user`: son los datos auxiliares de usuario que se han pasado a `pcap_loop`
- `pkt_header`: es un objeto de tipo `pcap_pkthdr` que contiene la cabecera pcap del paquete leído o capturado. Este objeto tiene tres campos:
 - `pkt_header.ts`: objeto `timestamp` que contiene el tiempo de captura del paquete. A su vez este objeto tiene 2 campos:
 - `ts.tv_sec`: timestamp del paquete en segundos.
 - `ts.tv_usec`: microsegundos dentro del segundo actual de timestamp. **OJO: Este valor nunca debe ser superior a 1,000,000. Esta variable NO almacena el mismo tiempo que `tv_sec` pero en microsegundos sino la parte fraccional del tiempo de captura.**
 - `pkt_header.len`: longitud real del paquete.
 - `pkt_header.caplen`: longitud capturada del paquete. Esto es, `pkt_data` solo contendrá `pkt_header.caplen` bytes.
- `pkt_data` es un `bytearray` que contiene los datos del paquete en caso de éxito.
- Es responsabilidad de `pcap_loop()`, no de la función de atención, reservar y liberar la memoria para devolver la cabecera y datos de cada paquete.

Hay otras funciones para leer paquetes no basadas en bucles, del tipo `pcap_next_ex()/pcap_next()`. **No las use de ninguna manera en las prácticas. Los motivos son de tipo docente. Su uso conllevará una evaluación nula.**

Guardar archivo pcap

Para guardar un archivo pcap necesitamos primero crear el archivo donde vamos a ir volcando los paquetes. Para ello se usan las funciones `pcap_open_dead` y `pcap_dump_open`:

`pcap_open_dead(linktype, snaplen)`

Donde

- `linktype` es el tipo de enlace de los paquetes que vamos a guardar. Típicamente, para redes Ethernet: `DLT_EN10MB`
- `snaplen` es el tamaño máximo que queramos guardar de cada paquete. Típicamente, para redes Ethernet, 1514 Bytes (ojo: puede haber `jumboframes`, más grandes).

Devuelve, como las otras funciones `pcap_open`, un descriptor de archivo pcap.

Ejemplo: `descr2 = pcap_open_dead(DLT_EN10MB, 1514)` # Abre un descriptor de archivo pcap para paquetes Ethernet, guardando como máximo 1514 Bytes de cada paquete.

`pcap_dump_open(descr, fname)`

Donde

- *descr* es el descriptor de archivo pcap previamente abierto con *pcap_open_dead*.
- *fname* es una cadena con el nombre del archivo pcap en el que queramos guardar los paquetes.

Devuelve un objeto *dumper* que se usará para guardar paquetes.

Ejemplos: *pdumper = pcap_dump_open(descr2,'salida.pcap')*

Crea un archivo llamado *salida.pcap* con las características (tipo de enlace, y tamaño máximo de paquete) de *descr2* (que indicamos en el *pcap_open_dead*). Nos devuelve un objeto *dumper*. Para guardar paquetes en el archivo creado con *pcap_dump_open* usamos la función:

pcap_dump(dumper,h,sp)

- *dumper* es el *dumper* devuelto por *pcap_dump_open*.
- *h* es un objeto de tipo *pcap_pkthdr* con la cabecera pcap del paquete que vamos a guardar.
- *sp* es un bytearray con el contenido del paquete. Se van a guardar tantos bytes como indiquemos en el campo *caplen* del parámetro *h*.

Ejemplo: *pcap_dump(pdumper,h,pkt_data)*

Guardamos en *pdumper* el paquete apuntado por *packet* con cabecera *h*.

Cerrar archivo

Los descriptors abiertos con *pcap_open_live*, *pcap_open_offline* y *pcap_open_dead* se cierran con *pcap_close(·)*. Mientras que los archivos abiertos con *pcap_dump_open* se cierran con *pcap_dump_close(·)*.

pcap_close(descr)

Donde

- *descr* es el descriptor a cerrar

pcap_dump_close(descr)

- *descr* es el *dumper* a cerrar

Enviar tráfico:

Para enviar tráfico usaremos la función *pcap_inject*.

pcap_inject(descr, buf,size)

Donde

- *descr* es el descriptor pcap abierto con *pcap_open_live*.
- *buf* es un bytearray con el contenido de la trama a enviar
- *size* es un entero que representa el número de bytes que tiene la trama a enviar

Devuelve un entero que representa el número de bytes enviados. Para comprobar el correcto retorno de esta función se aconseja comparar si el retorno es igual a *size*.

Ejemplos: *ret = pcap_inject(descr,trama,len(trama))*

Ejercicios

Libpcap

Se facilita un programa ejemplo en el Moodle. Descárguelo, analícelo y modifíquelo para que cumpla los requisitos definidos a continuación. **NOTA IMPORTANTE: El programa de ejemplo no cumple necesariamente con todos los requisitos del ejercicio. Es responsabilidad de los estudiante realizar las modificaciones que estimen oportunas para cumplir con todo lo requerido a continuación.**

Entregue los fuentes Python (*.py, incluyendo *rc1_pcap.py*) usados para implementar un programa basado en libpcap que:

1. Si se ejecuta **sin** argumentos, debe devolver ayuda de ejecución.
2. Si se ejecuta con el argumento **--itf**, consideramos que queremos capturar de interfaz:
 - El programa debe mostrar el número de paquetes recibidos por la interfaz de red especificada en el argumento tras pulsar Control-C o al finalizar el programa y la diferencia de tiempo entre el último y primer paquete capturado (siempre que haya al menos 2 paquetes, la diferencia será 0 en caso contrario) según la cabecera pcap.
 - El programa debe almacenar los paquetes capturados **enteros** en dos archivos de trazas con los siguientes nombres:
 - traza con nombre `capturaNOIP.nombreitf.FECHA.pcap` (donde *FECHA* será el tiempo actual UNIX en segundos y *nombreitf* el nombre de la interfaz especificada). En este fichero se almacenarán todos los paquetes cuyo valor de los bytes 12 y 13 (comenzando a contar en 0) de la trama capturada **NO** se corresponden con el valor 0x08 (byte 12) y 0x00 (byte 13), respectivamente.
 - traza con nombre `captura.nombreitf.FECHA.pcap` (donde *FECHA* será el tiempo actual UNIX en segundos y *nombreitf* el nombre de la interfaz especificada). En este fichero se almacenarán el resto de paquetes que no coinciden con la comprobación anterior.
1. Si se ejecuta con el argumento **--file**, consideramos que queremos analizar una traza pcap. El programa debe mostrar el número de paquetes de la traza al finalizar su ejecución y la diferencia de tiempo entre el último y primer paquete leído (siempre que haya al menos 2 paquetes, la diferencia será 0 en caso contrario) según la cabecera pcap.
2. En ambos casos (traza o captura de interfaz/en vivo) el programa debe **mostrar los N** (siendo N especificado en el parámetro **--nbytes**) primeros bytes de cada paquete capturado/analizado en hexadecimal con 2 dígitos por Byte en mayúsculas (**y separando cada Byte por espacios en blanco**), en líneas de hasta 16 bytes. Si el parámetro **--nbytes** no está establecido, se mostrarán todos los bytes de la trama.
 - Prestad atención a los límites de bytes capturados, y a paquetes más pequeños (¿los hay?).
 - Para demostrar la corrección de este tercer apartado use Wireshark: compare visualmente si la salida de su programa coincide con la salida que da Wireshark en su ventana inferior. Se espera que no haya diferencias.
 - Haga una captura de pantalla que muestre ambas salidas para una captura en vivo (*online*). Llame a la imagen de esta captura de pantalla `practica1captura.png`, e inclúyala en la entrega.
- 5.- En ambos casos (traza o captura de interfaz/en vivo) el programa debe procesar los primeros **P** paquetes, siendo P el número de paquetes especificado por la opción **--npkts**. Si este parametro esta establecido se capturará el número de paquetes especificado. Si el parametro no esta especificado el sistema deberá capturar paquetes de forma indefinida.

NOTA: El programa solo debe distinguir la fuente de entrada a la hora de abrir el descriptor: NO se debe por tanto hacer dos "flujos" distintos para cada tipo de operación sino tan solo un flujo que al principio distinga de donde "sale" el tráfico sobre el que trabajar pero, a partir de ese punto, debe haber un único flujo con las mínimas variaciones posibles.

NOTA: Los valores establecidos como "0xNúmero" se corresponden con el valor entero en notación hexadecimal (base16, "0-F").

Criterios de evaluación

Ejercicios: Entrega según el calendario.

- Normativa de entrega cumplida en su totalidad: 5%
- Fichero `leeme.txt` bien explicado: 5%
- Contar paquetes de una traza (independientemente del número de paquetes): 15%
- Contar paquetes de la interfaz de red: 10%
- Uso de un único "flujo" para traza e interfaz: 10%
- Almacenar correctamente el tráfico capturado en vivo una traza: 5%
- Separar correctamente el trafico capturado en dos trazas: 10%
- Gestionar correctamente el numero de paquetes a leer 10%
- Calcular tiempo entre último y primer paquete: 10%
- Imprimir los N primeros bytes de un paquete (pruebe para $N > 32$) y validarlo con Wireshark (captura de pantalla): 20%

Cuestionario individual: Se realizará un cuestionario para evaluar la comprensión del enunciado de la práctica vía Moodle el día indicado en el calendario de la asignatura.

Control individual: Control a través de Moodle sobre manejo básico de Wireshark y libpcap el día indicado en el calendario de la asignatura.

Entrega

Como regla general la entrega de las practicas deben ser autocontenidas, es decir, en el fichero de entrega se deberá incluir todos los ficheros necesarios para la correcta ejecución de la misma.

Respecto al ejercicio **Libpcap**, se deberán entregar **todos** los ficheros **.py** necesarios para la ejecución de la practica (incluido el `wrapper rc1_pcap.py`), siendo el archivo de entrega con nombre **practica1.py** donde el alumno deberá realizar los cambios

requeridos.

- Añada un archivo **leeme.txt** que incluya los nombres de los autores, comentarios que se quieran transmitir al profesor y, en caso de entregar algún archivo más, la descripción y/o explicación del mismo. **Además este fichero debe contener una sección donde se determine si se ha dado respuesta (Realizado/Parcialmente-Realizado/No-Realizado, y en caso afirmativo la explicación de cómo se ha validado) a cada criterio de evaluación solicitado. Ejemplo:**
 - **Normativa de entrega cumplida en su totalidad: Realizado: Varias relecturas del enunciado y revisión del archivo subido a Moodle.**
 - **Contar paquetes de una traza: Realizado: Se ha comprobado que el número de paquete que muestra nuestro programa coincide con el que indica Wireshark.**
 - **Contar paquetes de la interfaz de red: No-Realizado.**
 - **Almacenar en una traza el tráfico capturado en vivo: Realizado: Se ha comprobado que todos los bytes de la traza capturado coincide con lo que indica Wireshark en un conjunto diverso de paquetes.**
 - **Calcular tiempo entre primer y último paquete: No-Realizado.**
 - **Imprimir los N primeros bytes de un paquete y validarlo con Wireshark (captura de pantalla): Parcialmente-Realizado: Se imprimen correctamente solamente los 5 primeros bytes.**

Comprima en un zip **TODO** lo que vaya a entregar y llámelo practica1_YYYY_PXX.zip, donde YYYY es el grupo al que pertenece (miercoles1, miercoles2, viernes1, viernes2, viernes3), y XX (y solo XX) es el número de pareja (**con dos dígitos**) sin espacios..

Por ejemplo, para la pareja 5 del grupo miércoles 1: **\$ zip practica1_miercoles1_P05.zip ***

Importante, el nombre del fichero no debe contener espacios.

Solo es necesario que suba la entrega un miembro de la pareja.

Última modificación: viernes, 20 de septiembre de 2024, 13:40

Actividad previa

[Más ejemplos Python](#)

Ir a...

Siguiente actividad

[Material P1 \(V3\)](#)

[Política de privacidad](#)

[Contacto](#)

[Aviso legal](#)

[Accesibilidad](#)

[UAM Covid 19](#)